



CONFIDENTIAL — INTERNAL REPORT

Platform Transformation Report 2025-2026

Architecture, Performance & Capability Improvements
Graphite Insurance Management System

Prepared for: Alpha Direct Insurance — Executive & Operations Team

Date: March 2026

Scope: Full-stack platform modernisation (Backend · Frontend · Agents)

Baseline: Legacy Graphite monolithic system (pre-2025)

Current state: 3-Tier distributed architecture

*This document summarises architectural and operational improvements made to the Graphite platform.
Percentage improvements are based on infrastructure analysis and benchmarked estimates.*

01 — Architecture Transformation

Monolithic → 3-Tier Distributed

The old Graphite system was a monolith — everything (website, admin tools, customer data, business logic, background jobs) ran inside one application on one server. One bug, one traffic spike, or one slow query could affect every user and every function simultaneously. The new architecture separates the system into three independent tiers that communicate via secure APIs.



Tier 1 — Graphite Backend

Handles all business logic: policies, payments, claims, customer data, and database operations. Runs on the server. Never exposed directly to end users. Protected behind API authentication. Changes here do not affect the staff portal UI.

Server — Laravel PHP



Tier 2 — React Staff Portal

The staff-facing web application. Runs entirely in the browser. Talks to the backend via a versioned, secure API. Can be updated, redesigned, or replaced without touching the backend or causing any downtime for running transactions.

Browser — React + TypeScript



Tier 3 — Agent / Cron Server

A dedicated background workhorse. Runs scheduled tasks: payment reconciliation, AML screening, renewal processing, bulk imports, and AI-assisted data checks — all completely separate from the web servers so they never compete for resources.

Background — Python + PHP Artisan

What this means in practice: A bug in the staff portal cannot crash payment processing. A slow overnight reconciliation job cannot freeze the website. Each tier can be scaled, updated, or restarted independently — minimising risk and downtime.

BEFORE VS. AFTER — ARCHITECTURE

BEFORE

Single monolithic PHP application. Staff portal, payment webhooks, background jobs, and customer data all running in the same codebase. One deployment takes everything down. One slow query affects all users.

AFTER

3 independent tiers with versioned API contracts between them. Frontend, backend, and agent server deploy independently. A failure or update in one layer does not affect the others. Read traffic separated from write traffic at the database level.

BEFORE — API

One unversioned API (1,168 lines). Mobile apps, third-party webhooks, and the staff portal all shared the same routes. Any internal refactor risked breaking external integrations.

AFTER — API

Dedicated versioned API v1 (93 clean endpoints with typed serialisers) for the React portal. Legacy routes preserved intact for third-party webhooks (DPO, VCS, WhatsApp). No breakage on either side.

02 — Database Load & Speed

Read Replicas · Redis Cache · Query Optimisation

65%

Database Load Reduced

Read traffic moved to replica servers. Primary database now only handles writes — policy creation, payments, updates. Staff browsing reports no longer competes with live transactions.

80%

Database Reads Eliminated

Redis cache absorbs ~80% of all read requests. Product catalogues, policy details, customer records served from memory in under 1 millisecond instead of querying the database every time.

5×

Policy Search Speed

Policy lookups now use numeric IDs and pre-computed database views instead of text string matching. Equivalent to an indexed phonebook vs. reading every page to find a name.

60%

Page Load Time Reduced

Redis cache responds in <1ms. Previously 5–15ms database wait on every page load, multiplied across every user at once. At peak hours, the difference is even greater.

99%

Bulk Op Response Time

Imports and batch operations that previously blocked the UI for 30–120 seconds now return a confirmation in under 1 second. The actual work runs silently in the background queue.

5×

Concurrent User Capacity

~50 concurrent users previously caused noticeable slowdowns. Infrastructure now supports 250+ concurrent users stably, with read replicas absorbing the additional load.

HOW THE SPEED IMPROVEMENTS WERE MADE

- ✓ **Read/Write Replica Split** — Two separate database servers. All staff browsing, searches, and reports go to the read replica. Only actual data changes hit the primary server. **65% load reduction**
- ✓ **Redis Cache with Smart Expiry** — Four cache levels: 5 minutes for live queries, 30 minutes for policies and customers, 1 hour for product data, 24 hours for reference data. Tagged invalidation means only the changed data is cleared, not everything. **80% fewer DB reads**
- ✓ **6 Pre-Computed Database Views** — Complex joins (policy + customer + payments + vehicles + KYC) are pre-assembled at the database level. Staff portal queries pull from these views instantly instead of assembling them from scratch on every request.
- ✓ **Integer ID Migration** — Policy references migrated from text strings ("MIS2021020969") to numeric IDs. Database indexes work on numbers, not strings. Every policy lookup is now a direct index hit. **5× faster searches**
- ✓ **Background Job Queue** — 14 background job classes handle: PDF generation, bulk imports, AML screening, email/SMS notifications. These no longer block HTTP responses. **99% faster for users**

- ✓ **5 Separate Database Connections** — Main read replica, primary write, AI tables write-only, archive database, logs database. Each type of traffic is isolated so a slow log write never delays a policy lookup.

03 — Security

Encryption · 2FA · Credential Vault · Compliance

AES-256

Credential Encryption

All API keys (payment gateways, SMS, cloud storage) are now encrypted using bank-grade AES-256 encryption. Previously stored as plain text in a server config file anyone with SSH access could read.

100%

Staff Accounts with 2FA

Two-factor authentication is now mandatory for all admin accounts via Laravel Fortify. A stolen password alone cannot grant access. Previously: password only — zero second factor.

99%

Faster Credential Rotation

Rotating an API key (e.g. after a suspected leak) previously required an IT engineer, server access, file edit, and restart — 1-2 hours. Now: Admin Panel → Vault → new key → Save. 10 seconds.

ENCRYPTED CREDENTIALS VAULT

BEFORE — PLAIN TEXT

All 30+ API keys for DPO, RealPay, Flutterwave, Infobip, AWS, OpenSanctions, and others stored as plain text in .env file on the server. Any developer or engineer with SSH access could read every secret. No access log, no rotation workflow, no audit trail. Changing a key required a server restart causing brief downtime.

AFTER — ENCRYPTED VAULT

34 credentials across 7 categories (AI, Payments, SMS, Cloud, Compliance, Tracking, Email) stored using AES-256 encryption. Vault requires a 6-digit PIN to unlock. 3 wrong attempts → 30-minute lockout. Auto-locks after 15 minutes of inactivity. Managed from admin panel browser UI — no server access needed. Changes live within 10 seconds.

SECURITY IMPROVEMENTS IN FULL

- ✓ **Mandatory 2FA on all staff accounts** — Fortify two-factor authentication with password confirmation. Account takeover risk reduced by ~90% — a stolen password is useless without the second factor. ~90% risk reduction
- ✓ **Brute-force lockout** — 3 wrong Vault PIN attempts triggers a 30-minute lockout. Login attempts are rate-limited separately. Automated blocking of credential-stuffing attacks.
- ✓ **XSS input sanitisation middleware** — All incoming user input is sanitised before it reaches the application layer. Cross-site scripting attacks blocked at the middleware level.
- ✓ **Customer PII masking in transit** — Re-KYC personal data goes through dedicated encryption and masking middleware before leaving the server. ID numbers and sensitive fields masked in API responses.
- ✓ **AML sanctions screening — now automated** — Every customer checked against global sanctions databases (OpenSanctions) in real time, covering name, date of birth, and nationality. Previously a manual periodic process. Missed hits carry regulatory risk and potential fines.
- ✓ **API key per-operation rate limiting** — Read and write API operations have separate rate limiters. A runaway read query cannot exhaust the write quota and vice versa.
- ✓ **Sanctum token isolation** — API tokens stored separately from the main database (SQLite). Token writes never touch the read-replica, preventing authentication failures under load.

- ✓ **CORS locked to known origins** — Only the production React portal domain and localhost (development) are permitted. All other origins blocked at the API gateway level.
-
- ✓ **Registration endpoint disabled** — No public sign-up route exists. All accounts are provisioned by administrators only. Eliminates a common attack surface.

04 — Payment Reconciliation & Anomaly Detection

From Reactive to Proactive

The Problem: Finance discovered payment problems only when customers called to complain — cancelled policies still deducting, premium increases not reflected in debit orders, partial payments going unnoticed. By the time an issue was found it had often run for weeks or months.

6

Automated Checks Daily

Six reconciliation checks run every morning at 6 AM, before the business day starts. Issues are flagged and waiting for the team when they open their dashboard — not discovered via customer complaints.

15-20h

Weekly Hours Saved

Finance team no longer manually cross-checks RealPay statements against policy premiums, cancelled policy lists, and payment schedules. The system does this automatically every night.

7

Payment Gateways Integrated

DPO, RealPay, VCS, Flutterwave, N-Genius, Orange Money, and Infobip — all fully integrated with reconciliation. Previously 4 partial integrations with no cross-gateway reconciliation.

THE 6 AUTOMATED RECONCILIATION CHECKS

- ✓ **Cancelled Policy Still Collecting** — Every cancelled policy is checked against active RealPay debit order contracts. If a cancelled policy has an active contract, it is flagged as critical. *(Root cause of complaints like MIS2019000047.)* **Critical**
- ✓ **Premium vs. Debit Order Mismatch** — When a premium increases (e.g. renewal adjustment), the RealPay instalment amount must match. Mismatches are flagged — the customer would otherwise be undercharged or overcharged. **High**
- ✓ **Partial Payments** — Payments received that are more than P5 below the expected premium are flagged. Identifies customers who may be accumulating shortfalls unnoticed. **Medium**
- ✓ **Unpaid Invoices** — Scheduled payment transactions that were never collected and are more than 24 hours overdue. Surfaces failed debit runs that need follow-up. **Medium**
- ✓ **Accumulating Balance** — Policies where total payments received are more than 20% below the total expected over the policy term. Identifies customers drifting into arrears without triggering a single large failure. **High**
- ✓ **Payment Gap** — Active policies with no successful payment in more than 60 days. May indicate a silently failed debit order or a customer who has switched banks. **High**

ANOMALY MANAGEMENT WORKFLOW

BEFORE

Reactive. Customer calls → finance investigates → discovers cancelled policy still deducting → escalates to IT → IT checks RealPay → cancels contract. Average resolution: 3-7 days after complaint. Customer trust already damaged.

AFTER

Proactive. Agent runs at 6 AM → anomaly logged with severity (critical/high/medium/low) → ops team sees it on dashboard → acknowledge → resolve with notes. Auto-resolves if condition clears itself. Full audit trail. Customer never needs to call.

05 — AI Assistant

Natural Language Data Queries

180x

Faster to Get Answers

A question like "How much did we collect in March by product?" previously took 30+ minutes (login → navigate → filter → export → Excel pivot). Now: type the question → answer in under 10 seconds.

7

Data Query Tools

The AI has 7 tools: search policies, payment summaries, customer lookups, claims, anomalies, statistics, and custom SQL queries. It chooses the right tool automatically based on the question.

2

AI Providers Supported

Groq (Llama 3.3 70B — free tier, fast) and Anthropic Claude (most capable, paid). Switchable from the admin panel in 30 seconds without any code change or deployment.

WHAT STAFF CAN ASK THE AI

- ✓ "How many active policies do we have?" → *75,034 active*
- ✓ "Total collections this month by product" → *P 5.2M breakdown*
- ✓ "Which customers haven't paid in 60 days?" → *policy list with gaps*
- ✓ "Show cancelled policies still collecting" → *anomaly list*
- ✓ "New policies vs cancellations this month" → *trend table*
- ✓ "Top 10 customers by premium value" → *ranked list*
- ✓ "Collection rate by payment method" → *breakdown with export*
- ✓ "Premium mismatches for motor policies" → *discrepancy list*

BEFORE VS. AFTER — GETTING A BUSINESS ANSWER

BEFORE — MANUAL PROCESS

Login to admin → navigate to Finance Dashboard → set date filters → wait for page to load → export data → open in Excel → build a pivot table → format numbers. 30+ minutes. Requires Excel knowledge. Can only answer questions that a pre-built report covers.

AFTER — AI ASSISTANT

Open AI Assistant → type question in plain English → get answer with data table in under 10 seconds. Can answer any question across policies, payments, claims, and customers. Results exportable to CSV. Conversation history saved per user. No Excel, no filters, no navigation.

Cost note: The AI runs on Groq's free tier (Llama 3.3 70B) — zero API cost for everyday queries. The Anthropic Claude option is available for more complex analysis when needed. Both are configurable from the admin panel without touching the server.

06 — Summary Scorecard

Before vs. After — At a Glance

What Changed	Before	After	Improvement
System Architecture	Single monolithic application	3-tier: Backend / Portal / Agents	Fully Decoupled
Database Read Load	100% on primary database	~35% on primary (read replicas)	65% Reduction
Redundant DB Reads	Every page load hits database	80-95% served from cache	~80% Eliminated
Policy Search Speed	Text string scan	Integer index + DB views	~5× Faster
Page Load Time	5-15ms database wait	<1ms cache response	~60% Faster
Bulk Operation Response	30-120 seconds blocking	<1 second (queued async)	99% Faster
Concurrent User Capacity	~50 (degraded performance)	250+ (stable)	5× Capacity
API Credential Security	Plain text in server file	AES-256 encrypted Vault	Critical → Secure
Staff Account 2FA	Password only	Mandatory 2FA (Fortify)	0% → 100%
Credential Rotation Time	1-2 hours (engineer + restart)	10 seconds (admin panel)	99% Faster
Payment Anomaly Detection	Days-weeks (reactive)	Next morning (automated)	Proactive
Reconciliation Checks	0 (manual)	6 checks every morning	0 → 6/day
Finance Manual Hours (recon)	15-20 hours/week	Near zero (automated)	~90% Saved
Business Query Time	30+ minutes (manual Excel)	<10 seconds (AI assistant)	180× Faster
Payment Gateway Coverage	4 (partial)	7 (fully integrated)	75% More
AML / Sanctions Screening	Manual periodic	Real-time automated	Manual → Automated
API Versioning	None — single shared API	v1 with typed serialisers	Future-Proof

PLATFORM FIGURES (LIVE)

- 75,034 active policies
- P 5.2M collected March 2026
- 1,944 API routes
- 333 database migrations
- 216 automated commands
- 7 payment gateways
- 34 encrypted credentials
- 5 database connections